

Observability

quinta-feira, 24 de abril de 2025 07:13

This project aims to qualify each API b running a set of automated tests and saving their results

So far the proposed organization relies on a new container with python (because want to practice that language) which will make all the requests when the server is running

So far the idea is to use pytest for that, but let's see

Proposed fodler architecture relies on

```
your-project-root/
├── docker/
│   ├── dev.Dockerfile      # For your C++ dev container
│   └── observability.Dockerfile
├── py-observability/
│   ├── Dockerfile          # For Python
│   ├── main.py             # Python script to test API
│   └── requirements.txt     # Python dependencies
└── docker-compose.yml
```

4. About your idea: **cache results and save later**

You wrote:

"Maybe a local cache to store data while tests are running"

Yes!

You could create an in-memory Python object (like a `list`) to collect **metrics**, and then at the **end of the tests** (maybe using `pytest` `hooks`) write everything into a database or file.

Sketch idea:

```
python Copy Edit

test_metrics = []

def test_something():
    start = time.time()
    # run test
    elapsed = time.time() - start
    test_metrics.append({
        "test_name": "test_something",
        "elapsed_time": elapsed
    })

# Later, save test_metrics somewhere
```

Or you can use `pytest_sessionfinish(session, exitstatus)` to hook at the end and save the metrics.

2. Manually measure inside the test (Python timer)

If you want to measure *specific code* manually (not the entire test), you can do:

```
python                                                                    Copy Edit

import time
import requests
import pytest

from configs.variables import *

def test_v1_listFilesInUrl():
    input_json = { "url": "some_url" }

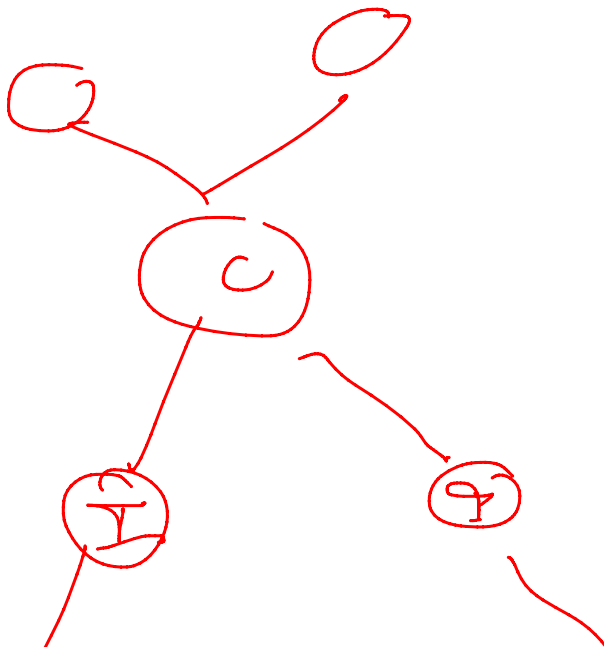
    start_time = time.time()

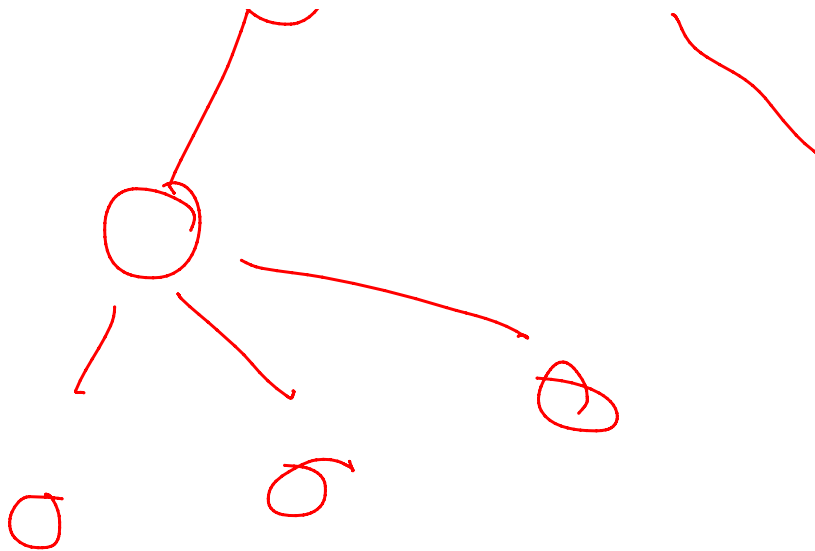
    response = requests.post(f"{BASE_URL}/api/v1/listFilesInUrl", json=input_json)

    elapsed = time.time() - start_time
    print(f"Request took {elapsed:.4f} seconds")

    assert response.status_code == 200
```

- `time.time()` gives you the time in seconds
- You subtract to get the **elapsed** time.





In Python, the two most common frameworks for writing and running unit tests are unittest (a built-in module) and pytest (a popular third-party library).

Using unittest

The unittest module is included with Python's standard library. To use it, you create a test file (conventionally named `test_something.py`) with a class that inherits from `unittest.TestCase`. Test methods within the class must start with `test_`.

To run tests with unittest:

From the command line, navigate to the directory containing your test file and run the following command:

Bash

```
python -m unittest
```

This command automatically discovers and runs all test files (files whose names start with `test_`) and test methods within them.

To run a specific test file, you can specify its name:

Bash

```
python -m unittest test_my_module
```

To run a specific test method within a test class:

Bash

```
python -m unittest test_my_module.MyTestClass.test_my_method
```

Using pytest

pytest is a widely-used and highly-regarded third-party testing framework. It simplifies a lot of the boilerplate code required by unittest and offers a more readable style. You must install it first:

Bash

```
pip install pytest
```

To use pytest, you write functions (or methods in a class) that start with `test_`. Assertions are made using the standard `assert` statement.

To run tests with pytest:

From the command line, simply navigate to your project directory and run:

Bash

```
pytest
```

pytest automatically discovers test files (files that start with `test_` or end with `_test.py`) and test functions or methods within them. It provides a detailed, color-coded output that is easy to read.

When choosing a framework, pytest is generally recommended for new projects due to its simplicity and powerful features, while unittest is a solid choice when you don't want to add external dependencies.